

ARQUITETURA TÉCNICA · AGOSTO 2026

Como a Match House está construída.

O Smart Link com IA para corretores de imóveis: a landing page, a camada de medição e atribuição, o pipeline de deploy — e onde estão as fronteiras do sistema.

REPOSITÓRIO

match-house/
lp-match-house

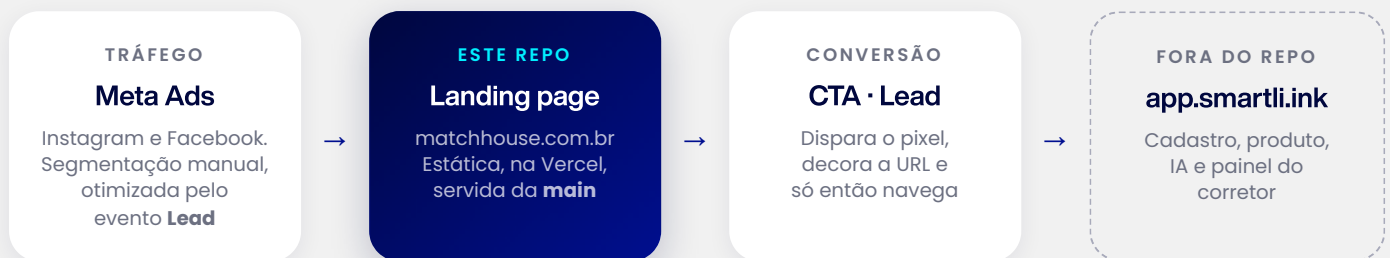
PRODUÇÃO

www.matchhouse.com.br
app.smartli.ink

ESCOPO DESTES DOCS

Aquisição: LP,
tracking e deploy

A Match House é uma **plataforma de tecnologia** para corretores de imóveis de alto padrão — não é imobiliária, portal, nem geradora de leads. O produto é o **Smart Link**: um único link que reúne imóveis, redes e WhatsApp do corretor, com uma IA que atende, qualifica e agenda 24/7 em nome dele. Este documento cobre a **camada de aquisição**, que é o que vive neste repositório.



O QUE ESTÁ NESTE REPOSITÓRIO

- **index.html** — a LP publicada, uma página só
- **tracking.js** — pixel, GA4, atribuição e eventos
- **/diagnostico** — quiz de 4 passos que termina no WhatsApp
- **politica-de-privacidade.html** — termos e LGPD
- **_ds/** — design system da marca (tokens, fontes, kits)
- **vendor/** — React e Phosphor auto-hospedados
- **anuncios/** — criativos de campanha versionados
- **concept-a...d.html** — rascunhos, não publicados

O QUE NÃO ESTÁ — E POR QUÊ

A fronteira do sistema é o clique no CTA. Tudo depois dele vive em outro domínio e outro repositório:

- O **cadastro** e a conta do corretor
- A **Match IA** — atendimento, qualificação, geração de descrição
- O **painel de performance** e a gestão de imóveis
- Os **apps** iOS e Android

CONTRATO ENTRE AS DUAS PONTAS

A LP entrega ao app a atribuição completa via query string — UTMs, `fbclid`, `gclid` e os cookies `_fbp` / `_fbc`. É assim que uma venda lá na frente volta a apontar para o criativo que a gerou.

1

página HTML em produção — a LP inteira é um arquivo só, sem rotas

0

passos de build. Sem npm, sem bundler, sem CI: o que está no git é o que roda

3

sistemas de medição em paralelo: Meta Pixel, GA4 e Vercel Analytics

A decisão estruturante: **site estático, sem etapa de build**. O HTML no repositório é literalmente o HTML que o navegador recebe. Isso mantém o custo de publicar em segundos e o diff legível — mas o preço é que a LP usa um runtime próprio no lugar de um framework tradicional.

O runtime `<x-dc>` — “design doc”

A LP não é HTML comum. O corpo da página é um template dentro de `<x-dc>`, compilado em tempo de execução por `support.js` (um runtime gerado — o arquivo é build output, não se edita à mão). Ele monta React a partir do template e de uma classe de lógica no mesmo arquivo.

<code>{{ expressão }}</code>	Interpolação de valores vindos de <code>renderVals()</code> , inclusive handlers de evento
<code><sc-for list as></code>	Repetição sobre arrays — usada nas notificações, no feed ao vivo e nos chips
<code><sc-if value></code>	Renderização condicional — cada etapa da conversa aparece por aqui
<code>style-hover</code>	Estado de hover inline, sem folha de estilo separada
<code><helmet></code>	Bloco de CSS e dependências injetado no <code><head></code> depois do load
<code>data-props</code>	Props ajustáveis por editor visual — hoje a velocidade de digitação (65 ms) e o número de contato

ZERO DEPENDÊNCIA DE CDN EM RUNTIME

O runtime busca React no unpkg por padrão. Se o unpkg estivesse lento ou bloqueado, **a página inteira renderizava em branco** — é o React que desenha todo o conteúdo. A correção usa o override oficial do próprio runtime:

```
window.__resources = {
  ".../react.production.min.js":
    "vendor/react.production.min.js",
  ".../react-dom.production.min.js":
    "vendor/react-dom.production.min.js"
};
```

React, React-DOM e os ícones Phosphor passaram a ser servidos do próprio domínio. Fontes da marca (Poppins e Helvetica Now Display) já eram auto-hospedadas. **Nenhum recurso crítico vem de terceiros.**

DESIGN SYSTEM VERSIONADO

A pasta `_ds/` carrega o brandbook traduzido em código: tokens CSS de cor, tipografia, sombra e motion, mais os UI kits. A paleta é enxuta de propósito — quatro cores, com o ciano como acento de no máximo 5%.

■ navy #00073e ■ blue #000f90 ■ cyan #00efff
□ paper #f1f1f1

ARMADILHA CONHECIDA

Favicon e `<title>` ficam no `<head>` estático real, nunca dentro do `<helmet>`. O helmet só injeta as tags via JS depois do load, e nesse caminho o navegador não atualiza o favicon de forma confiável. Foi um bug real, e a regra está fixada no `CLAUDE.md` do projeto.

SEIS SEÇÕES, UMA NARRATIVA

SEÇÃO	O QUE É	PAPEL NA CONVERSÃO
S1	Conversa com a IA	Primeira dobra. A IA digita, o visitante responde em chips e dá o nome — a página se personaliza na hora
S2	O que é o Smart Link	Explicação do produto: um link com imóveis, redes, WhatsApp e a Match IA
S3	Madrugada 23h47	Prova emocional: relógio correndo e notificações de leads chegando enquanto o corretor dorme
S4	Rede ao vivo	Feed de eventos da rede + contador de leads. Rotulado como painel ilustrativo
S5	Smart Links reais	Prova social com Smart Links de corretores no ar
S6	CTA final + rodapé	Último empurrão sobre o gradiente da marca

A conversa é uma máquina de estados

O coração da LP são seis passos numa classe que estende `DCLogic`. Cada passo tem um texto que é **digitado caractere a caractere** e um conjunto de respostas possíveis.

- 1 Apresentação** — a IA se apresenta; chips de continuar ou perguntar quem é ela
- 2 Captura do nome** — campo livre; aceita nome ou `@` do Instagram e normaliza
- 3 Dor** — três opções: lead frio, falta de tempo, link fraco
- 4 Resposta ramificada** — o texto muda conforme a dor escolhida
- 5 Prévia** — mostra o Smart Link com o nome e o slug do visitante
- 6 Fechamento** — preço ou cadastro, e o CTA final aparece

DETALHE QUE COSTUMA SURPREENDER

A partir do primeiro nome o código **infere o gênero** — por lista de nomes conhecidos, com fallback na terminação em “a” ou “o”. Isso escolhe o avatar da prévia e concorda o artigo: “o Bernardo IA” ou “a Marina IA”.

OS RELÓGIOS QUE DÃO VIDA À PÁGINA

INTERVALO	O QUE AVANÇA
65 ms	Um caractere da fala da IA <code>ajustável</code>
1 000 ms	O relógio da seção madrugada
1 600 ms	O contador de leads qualificados
2 100 ms	Uma linha nova no feed da rede
2 600 ms	Uma notificação nova na madrugada

Todos são limpos em `componentWillUnmount`. A velocidade de digitação é prop de editor: mexeu nela, mexeu no ritmo da primeira dobra — por isso está congelada em 65 ms.

RESPONSIVIDADE SEM MEDIA QUERY

Os grids usam `auto-fit` com `minmax(min(330px, 100%), 1fr)`: colapsam sozinhos no celular. Os tipos usam `clamp()` em vez de breakpoints. Como quase todo o tráfego vem de Instagram e Facebook, **mobile é o caso principal**, não a exceção.

A parte mais densa do repositório. `tracking.js` resolve quatro problemas que quebram medição em campanha paga — todos descobertos em produção, com dinheiro rodando.

PROBLEMA 1 · O CLIQUE MATAVA O PRÓPRIO EVENTO

Navegar cancela requisições em voo: o beacon do pixel saía junto com a navegação e **morria antes de chegar** — leads reais não apareciam no Meta.

t=0 Clique capturado por **listener delegado** na fase de captura — pega CTA renderizado depois. `preventDefault()` segura a navegação

+1ms Dispara **Meta Pixel** com um `eventID` único, e **GA4** com `event_callback`

~100ms GA4 confirma o recebimento → **navega imediatamente**

teto **400 ms** com GA4 no ar (rede de segurança) · **150 ms** se o GA4 estiver bloqueado

O segundo teto existe porque com adblock o callback *nunca* chega. Detectamos o gtag real por `window.google_tag_manager` — sem isso, o botão ficava 900 ms morto no dedo da pessoa.

PROBLEMA 2 · UTMS CORROMPIDOS PELO META

Anúncios foram salvos com a URL inteira colada no campo “Parâmetros de URL”. O Meta anexava aquilo à query e a página chegava com um par-lixo (`?https://...?utm_source=...`) — o GA4 perdia a origem da sessão. Antes de inicializar pixel e GA4, o script **resgata o `utm_source` de dentro do lixo, descarta o par quebrado, remove UTM duplicado e reescreve a URL com `replaceState`**, para os dois lerem uma query limpa.

PROBLEMA 3 · ATRIBUIÇÃO ETERNA

O corretor clica no anúncio, sai, e volta dias depois pelo direto. A atribuição fica em `localStorage` para sobreviver a isso — mas **com TTL de 30 dias**. Sem o TTL, um clique antigo valia para sempre e contaminava a leitura por criativo. O formato antigo, sem timestamp, é **descartado de propósito**: era ele que carregava o resíduo de campanhas pré-retag.

PROBLEMA 4 · A CONVERSÃO ACONTECE EM OUTRO DOMÍNIO

O cadastro vive em `app.smartli.ink`. Antes de navegar, a URL é decorada com a atribuição guardada e os cookies `_fbp` / `_fbc` — a ponte cross-domain que devolve a conversão ao criativo certo. Cada evento sai com um `eventID` único: a chave de deduplicação para quando a **Conversions API server-side** entrar.

EVENTO	DISPARA EM	ESPELHO NO GA4	PARA QUE SERVE
Lead	Todo CTA de cadastro	<code>signup_cta_click</code>	Otimização da campanha. Renomear zera o aprendizado — regra fixa do projeto
Contact	Links de WhatsApp	<code>whatsapp_click</code>	Só medição. Hoje ativo no /diagnostico; na LP todos os CTAs vão para o app
LPView	Carga da página	—	PageView customizado, carimbado com o <code>utm_content</code>
ScrollDepth	25 / 50 / 75 / 100%	<code>scroll_depth</code>	Onde a página perde a pessoa. Listener passivo, um disparo por marco

PASSO 1

commit

Alteração direta no HTML ou JS



PASSO 2

push na main

A **main** é a produção. Branch de feature não muda o site



PASSO 3

Vercel

Publica os arquivos estáticos. Sem build, sem container



PASSO 4

no ar

Validar com hard refresh — o cache é agressivo

CONFIGURAÇÃO DE PLATAFORMA

vercel.json

Redireciona `/go` para a raiz — a URL curta usada nos anúncios, mantida temporária de propósito

Vercel Analytics

`/_vercel/insights/script.js`, medição de audiência independente do GA4

Verificação Meta

Meta tag de domain verification no `<head>`

Local

`npx serve -p 3456 .` — nada além de um servidor de arquivos

PÁGINAS SATÉLITE

/diagnostico

Quiz de 4 passos que gera um diagnóstico e leva ao WhatsApp. Stack **diferente**: React por CDN + Babel no navegador, com pixel e conversão de Google Ads próprios

Política

Termos de uso e privacidade, HTML puro

concept-a...d

Rascunhos de conceito. Publicados no domínio, mas fora do fluxo

HIGIENE DE PERFORMANCE

- **React, React-DOM e Phosphor saíram do CDN** e passaram a ser servidos do próprio domínio — um terceiro fora do ar deixava a página em branco
- **O CTA é liberado mais cedo** na conversa, sem esperar a animação inteira terminar
- **Fontes auto-hospedadas** desde o início, sem chamada ao Google Fonts

DEPOIS DE PUBLICAR

O cache da borda é agressivo, e o do favicon mais ainda. A checagem depois de todo deploy:

- 1 Confirmar que o commit chegou na `main`
- 2 Abrir o site com **hard refresh** (Ctrl+Shift+R)
- 3 Conferir aba e favicon — foram o último bug de cache
- 4 Clicar num CTA e ver o `Lead` chegando no Meta

Regras que não se negociam

Cada uma custou dinheiro ou aprendizado de campanha. Estão fixadas no `CLAUDE.md` para não voltarem.

- **Não renomear o evento `Lead`**. A campanha otimiza por ele; trocar o nome zera o aprendizado do conjunto
- **Nada muda em campanha ativa sem aprovação** — verba, lance, público ou posicionamento
- **Nunca ligar expansão de público** (Advantage+). A segmentação é sempre manual — decisão explícita
- **Não regredir o que já foi corrigido**: velocidade de digitação e grids de mobile são território minado

Onde a arquitetura cobra o preço das próprias escolhas. Esta é a parte que vale discutir na mesa.

ESCOLHA	O QUE GANHAMOS	O QUE CUSTA
Site estático, sem build	Publicar em segundos; diff legível; nada quebra por dependência	Sem type check, sem lint, sem teste automatizado. A revisão é o único portão
Runtime <code><x-dc></code> próprio	React com estado real sem tocar em bundler	Ninguém de fora conhece o DSL. <code>support.js</code> é build output — não dá para corrigir no lugar
Tudo em um HTML	Uma leitura só entende a página inteira	~66 KB de arquivo, estilo inline, conflito garantido se duas pessoas editarem juntas
Medição só no cliente	Zero backend para manter	Adblock e Safari derrubam parte dos eventos. O <code>eventID</code> já está lá para a CAPI
<code>/diagnostico</code> com Babel no navegador	Foi ao ar rápido	Compila JSX no aparelho do visitante, e usa um pixel diferente da LP — duas verdades de medição

FILA TÉCNICA, EM ORDEM DE IMPACTO

- 1 **Conversions API server-side.** A dedup por `eventID` já está pronta; falta o endpoint. É o que recupera o evento perdido para adblock
- 2 **Unificar a medição do `/diagnostico`** com a da LP, ou assumir explicitamente que são dois funis separados
- 3 **Tirar o Babel do runtime** do `/diagnostico` — pré-compilar ou reescrever sem JSX
- 4 **Logo vetorial e ícones da marca.** Hoje: PNG raster e Phosphor como substituto declarado

PERGUNTAS PARA O CAFÉ

- Vale trocar o runtime próprio por algo padrão, ou o custo de migração come o ganho?
- Qual o mínimo de teste automatizado que faz sentido numa página só — smoke test de render e de CTA?
- CAPI: rota serverless na Vercel ou serviço à parte?
- A medição client-side está subestimando os leads em quanto, de fato?

Desbloqueie seu mundo.

www.matchhouse.com.br · app.smartli.ink